KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

Doubt Solving Bot for Engineering Students

Project ID: PRJ-2026-078
Team ID: T078

Team GitHub Repository: https://github.com/saiganesh90761/GenAi_Doubt_Solving_Bot/

Student Name	Registration Number	Individual GitHub Link
B SAI GANESH	99230040272	https://github.com/saiganesh90761/GEN-AI
K JASWANTH KUMAR	99230040317	https://github.com/jashuboss66-beep/genaimastery
U MAHA LAKSHMI	99230041170	https://github.com/saiganesh90761/GEN-AI
K NAGA PAVAN VARMA	99230040356	https://github.com/Nagapavanvarma-4444/GEN-AI
G KISHORE KUMAR REDDY	99230040312	https://github.com/kishorekumarreddy1803-maker/GEN-AI-Mastery
A SIVA KRISHNA	99230040258	https://github.com/sivakrishnaatkuri-commits/GEN-AI

Project Trainer / Guide
Mr. SHIRWIN PRINCE I

Table of Contents

Section	Topic
1	Introduction
2	Problem Statement and Objectives
3	System Architecture and RAG Workflow
4	Implementation Details and Technology Stack
5	Results and Testing

1. Introduction

The rapid development of Large Language Models (LLMs) has transformed the way students access and understand information. However, general-purpose AI assistants may generate inaccurate information, lack access to specific academic materials, and produce answers that are not directly supported by educational references.

Engineering students frequently refer to lengthy textbooks, lecture notes, technical documents, and academic PDFs to understand complex concepts and solve subject-related problems.

To address these challenges, this project introduces **Engineering Doubt Solver**, an AI-powered academic assistant based on Retrieval-Augmented Generation (RAG).

The system allows students to upload PDF documents and ask questions related to their contents. The uploaded documents are processed, divided into smaller text chunks, converted into vector embeddings, and stored in a vector database. When a student submits a question, the system retrieves semantically relevant document chunks and provides them as contextual information to a Large Language Model.

By combining semantic retrieval with generative AI, the application aims to provide context-aware academic explanations grounded in the student's uploaded learning materials.

2. Problem Statement and Objectives

Problem Statement:

Engineering students often spend considerable time searching through large textbooks, lecture notes, and academic PDFs to find explanations for specific doubts. Traditional keyword-based search systems are limited because they cannot understand the semantic meaning or context of natural-language questions.

Additionally, general-purpose AI chatbots may generate hallucinated answers or provide explanations that are not aligned with the student's reference materials.

Therefore, there is a need for an intelligent academic assistant that can understand uploaded documents, retrieve relevant information, and generate meaningful answers based on the available context.

Objectives:

- To develop an AI-powered academic assistant that allows students to resolve engineering-related doubts using their uploaded PDF documents.
- To implement a Retrieval-Augmented Generation pipeline for context-aware question answering.
- To convert document contents into vector embeddings for semantic search.
- To utilize Qdrant Cloud for efficient vector storage and retrieval.
- To integrate a Large Language Model through OpenRouter for generating natural-language explanations.
- To provide relevant document references and improve answer reliability.
- To develop a user-friendly web application with persistent data management.
- To deploy the application using a cloud-based deployment platform.

3. System Architecture and RAG Workflow

3.1 Data Ingestion Phase

The data ingestion phase converts uploaded academic documents into searchable vector representations.

The process consists of the following steps:

1. **PDF Upload:** Students upload their academic PDF documents through the web application.
2. **Text Extraction:** The application extracts textual content from uploaded PDFs using the existing PDF processing implementation.
3. **Text Chunking:** Extracted content is divided into smaller, manageable chunks to preserve contextual relevance and improve retrieval efficiency.
4. **Embedding Generation:** Each text chunk is converted into a numerical vector representation using the configured embedding model.
5. **Vector Storage:** The generated embeddings and associated metadata are stored in Qdrant Cloud.
6. **Document Metadata Management:** Supabase is used for managing relevant application data and document information.

   Each stored vector is associated with relevant metadata such as document identification and source information, allowing retrieved content to be associated with its original document.

3.2 Query Processing and Generation Phase

The query processing phase retrieves relevant academic information and generates an answer. The workflow consists of:

1. **User Query:** A student submits a natural-language academic question through the chat interface.
2. **Query Embedding:** The question is converted into a vector using the same embedding model used during document ingestion.
3. **Semantic Retrieval:** Qdrant Cloud performs vector similarity search to retrieve the most relevant document chunks.
4. **Context Construction:** The retrieved chunks are assembled into contextual information for the language model.

5. **LLM Generation:** The contextual information and student question are passed to the configured Large Language Model through OpenRouter.
6. **Response Generation:** The model generates a natural-language response based on the retrieved context.
7. **Response Delivery:** The generated answer is returned to the student through the web application's chat interface.

   The system aims to reduce unsupported responses by grounding AI-generated explanations in retrieved academic content.

4. Implementation Details and Technology Stack

The project uses a modern web application architecture combined with cloud-based AI and database services.

Component	Technology Used	Purpose
Frontend Framework	Next.js 16	Building the web application and user interface
Programming Language	TypeScript	Type-safe application development
Styling	Tailwind CSS	Responsive and consistent user interface
Backend	Next.js API Routes	Handling application logic and API requests
Database	Supabase	Managing persistent application data
Vector Database	Qdrant Cloud	Storing and retrieving vector embeddings
LLM Integration	OpenRouter	Connecting to the configured Large Language Model
PDF Processing	unpdf	Extracting text from PDF documents
Validation	Zod	Validating application inputs and data structures
Mathematical Rendering	KaTeX	Rendering mathematical expressions
Deployment	Vercel	Hosting and deploying the application

4.1 Sample Document Processing Configuration

The document processing pipeline uses text chunking to divide extracted PDF content into smaller sections before generating embeddings.

The chunking configuration determines the maximum size of each chunk and the overlap between consecutive chunks.

4.2 Vector Database Integration

Qdrant Cloud is used as the vector database for storing document embeddings.

Each vector is associated with metadata that identifies its source document and relevant information. When a user submits a question, the system performs similarity search to identify relevant chunks.

This approach enables semantic retrieval rather than relying exclusively on exact keyword matching.

4.3 Language Model Integration

OpenRouter provides access to the configured Large Language Model.
The retrieved document context is supplied to the model along with the user's question. The model uses this information to generate relevant academic explanations.

4.4 Database and Deployment

Supabase provides persistent database functionality for managing application-related information.
The application is deployed on Vercel, enabling users to access the web application through the internet without requiring a local development environment.

5. Results and Testing

The Engineering Doubt Solver has been developed as a web-based academic question-answering application.
The implemented workflow supports uploading academic PDF documents, processing their contents, generating embeddings, storing vectors in Qdrant Cloud, and retrieving relevant information to answer user questions.

Functional Testing:

The following functionalities are considered in the testing process:

Test Case	Expected Result
PDF Upload	Valid PDF is accepted
Text Extraction	Text is extracted from the uploaded document
Chunking	Extracted text is divided into chunks
Embedding Generation	Vector representations are generated
Vector Storage	Embeddings are stored in Qdrant
Semantic Retrieval	Relevant chunks are retrieved for a query
AI Response	Answer is generated using retrieved context
Web Deployment	Application is accessible through Vercel

Evaluation Metrics:

The following measurements can be used to evaluate the effectiveness of the system:

- Retrieval relevance.
- Answer correctness.
- Citation accuracy.
- Average response time.
- PDF processing time.
- Successful document indexing rate.

Quantitative evaluation results should be added after testing the application with a representative set of academic questions.

5. Industry Readiness, Project Outcomes and Future Scope

Conclusion:

The Engineering Doubt Solver project demonstrates the application of Retrieval-Augmented Generation in academic question answering.

By integrating PDF text extraction, embedding generation, vector similarity search, and Large Language Models, the system provides an interactive approach for students to resolve doubts using their own academic materials.

The use of Qdrant Cloud enables semantic retrieval of relevant document content, while Supabase

provides persistent database functionality. The integration of OpenRouter enables the generation of natural-language academic explanations.

The application aims to bridge the gap between static academic documents and interactive AI-powered learning.

Future Scope:

Future improvements to the system may include:

- Multimodal RAG to understand images, diagrams, charts, and mathematical expressions within PDFs.
- Hybrid search combining keyword and semantic retrieval.
- Reranking techniques to improve retrieval relevance.
- Advanced citation verification with page-level references.
- AI-generated quizzes and practice questions.
- Automated flashcard and revision-note generation.
- Voice-based academic doubt solving.
- Support for multiple languages.
- Personalized learning recommendations.
- Detailed learning analytics and progress tracking.
- Improved retrieval evaluation and hallucination detection.

These enhancements can transform the application into a comprehensive AI-powered academic learning platform